

UniApp 最常用自定义 tabbar 使用总结（极简版）

聚焦 uniApp 开发中自定义 tabbar 的高频用法，仅保留日常开发必用步骤、核心结构和避坑点，极简好记、可直接套用，贴合多端适配需求，延续之前总结逻辑。

一、核心前提（必做步骤）

自定义 tabbar 前，需先在全局配置中关闭默认 tabbar，同时配置 tabbar 基础信息（app.json 中配置）：

app.json 全局配置

```
1
2  {
3    "pages": [
4      "pages/index/index", // tabbar 对应页面1
5      "pages/mine/mine"    // tabbar 对应页面2（至少2个）
6    ],
7    "tabBar": {
8      "custom": true,      // 关闭默认 tabbar，开启自定义（核心）
9      "color": "#999",    // 未选中标签颜色（可选，自定义中可覆盖）
10     "selectedColor": "#333", // 选中标签颜色（可选）
11     "list": [            // 必须配置，对应每个 tab 页面（与自定义 tab 一一对应）
12       { "pagePath": "pages/index/index", "text": "首页" },
13       { "pagePath": "pages/mine/mine", "text": "我的" }
14     ]
15   }
16 }
17
```

说明：list 中配置的 pagePath 必须是 pages 数组中存在的页面，数量建议 2-5 个（多端适配最佳）。

二、最常用自定义 tabbar 结构（直接复用）

核心用 <view> 模拟 tabbar 容器，搭配 <view> 作为 tab 项，实现切换功能，适配多端，包含“图标+文本”（最常用场景）：

自定义 tabbar 组件（推荐全局组件）

```
1
2 <template>
```

```

3    <!-- 自定义 tabbar 容器（固定底部） -->
4    <view class="custom-tabbar">
5        <!-- 单个 tab 项（循环渲染，也可手动写） -->
6        <view
7            class="tab-item"
8            v-for="(item, index) in tabList"
9            :key="index"
10           @tap="switchTab(item.pagePath)"
11        >
12            <!-- tab 图标（选中/未选中切换） -->
13            <image
14                :src="currentPath === item.pagePath ? item.selectedIcon : item.icon"
15                class="tab-icon"
16            ></image>
17            <!-- tab 文本（选中/未选中颜色切换） -->
18            <text :class="{ 'tab-text-active': currentPath === item.pagePath }">
19                {{item.text}}</text>
20            </view>
21        </view>
22    </template>
23
24    <script>
25    export default {
26        data() {
27            return {
28                currentPath: '/pages/index/index', // 当前选中页面路径
29                // tab 列表（与 app.json 中 list 对应，可添加图标路径）
30                tabList: [
31                    {
32                        pagePath: '/pages/index/index',
33                        text: '首页',
34                        icon: '/static/tab/home.png', // 未选中图标
35                        selectedIcon: '/static/tab/home-active.png' // 选中图标
36                    },
37                    {
38                        pagePath: '/pages/mine/mine',
39                        text: '我的',
40                        icon: '/static/tab/mine.png',
41                        selectedIcon: '/static/tab/mine-active.png'
42                    }
43                ]
44            };
45        },
46        onShow() {
47            // 页面显示时，获取当前页面路径，更新选中状态（避免切换后状态错乱）
48            this.currentPath = getCurrentPages()[0].route;

```

```
49     methods: {
50         // tab 切换方法（核心）
51         switchTab(pagePath) {
52             uni.switchTab({ url: pagePath }); // 必须用 switchTab 跳转 tab 页面
53         }
54     }
55 };
56 </script>
57
58 <style scoped>
59 /* tabbar 核心样式（固定底部、占满宽度） */
60 .custom-tabbar {
61     position: fixed; /* 固定在底部，不随滚动移动 */
62     bottom: 0;
63     left: 0;
64     right: 0;
65     height: 50px; /* 固定高度，适配多端 */
66     display: flex;
67     justify-content: space-around; /* 均匀分布 tab 项 */
68     align-items: center; /* 垂直居中 */
69     background-color: #fff;
70     box-shadow: 0 -2rpx 10rpx rgba(0,0,0,0.1); /* 可选：顶部阴影，区分页面 */
71     z-index: 999; /* 避免被其他元素遮挡 */
72 }
73 /* 单个 tab 项样式 */
74 .tab-item {
75     display: flex;
76     flex-direction: column;
77     align-items: center;
78     justify-content: center;
79 }
80 /* tab 图标样式 */
81 .tab-icon {
82     width: 40rpx;
83     height: 40rpx;
84     margin-bottom: 4rpx;
85 }
86 /* tab 文本样式 */
87 .tab-item text {
```

三、高频核心操作（必记）

操作需求	实现方法
tab 页面切换	调用 uni.switchTab({ url: 页面路径 }), 不可用 navigateTo

更新选中状态	onShow 中获取当前页面路径，赋值给 currentPath，实现图标/文本切换
修改 tab 颜色	修改 .tab-item text（未选中）和 .tab-text-active（选中）的 color 属性
添加新 tab 项	1. app.json 中 tabBar.list 新增配置；2. 组件 tabList 中新增对应项

四、必看避坑注意事项（核心）

- 1. tabbar 用 `position: fixed` 固定后，页面底部内容会被遮挡，需给页面根容器加 `padding-bottom: 50px`（与 tabbar 高度一致）；
- 2. 跳转 tab 页面必须用 `uni.switchTab()`，不可用 `navigateTo`、`redirectTo`，否则无法切换 tab 状态；
- 3. app.json 中 `tabBar.list` 的 `pagePath` 必须与组件中 `tabList` 的 `pagePath` 完全一致（含 /），否则选中状态错乱；
- 4. 多端兼容：小程序、APP 需注意图标路径（建议用本地静态图标），H5 无特殊限制；避免用复杂样式（如渐变），防止多端显示异常；
- 5. 建议将 tabbar 封装为全局组件，在每个 tab 页面引入，避免重复写代码。

总结

- 1. 核心步骤：app.json 开启 `custom: true` + 配置 tab 列表 + 封装自定义 tabbar 组件；
 - 2. 常用结构：底部固定容器 + 循环渲染 tab 项（图标+文本）+ 切换方法；
 - 3. 避坑重点：解决底部遮挡、正确跳转方法、保持路径一致，直接复用示例即可满足80%场景。
- （注：文档部分内容可能由 AI 生成）